

Self-Learning Material

Program:MCA

Semester: 4

Course Name: Web APIs

Code: 21VMT3S402

Unit Name: Spring Boot Security

Table of Contents

Learning Outcome:	3
1. Securing Web Applications	4
Common Security Threats	4
2. Authentication and Authorization	6
Authentication	6
Authorization	7
3. Configuring Authorization and Resource Servers	7
Step 1: Set Up an Authorization Server	8
Step 2: Configure a Resource Server	9
Step 3: Implement Token-Based Authentication and Authorization	9
Step 4: Test the Configuration	10
4. JWT Tokens in Spring Boot Security	11
What is a JWT?	11
Using JWT Tokens in Spring Boot Security	11

UNIT 15: Spring Boot Security

Objectives:

In this Unit you will learn about –

- Common security threats and ways to prevent them.
- Authentication and authorization.
- Configuring Authorization and Resource Servers
- JWT Tokens

Learning Outcome:

Familiarize with the security aspects of Spring Boot

15. Spring Boot Security

1. Securing Web Applications

Web applications play a crucial role in our interconnected digital world, facilitating communication, commerce, and collaboration. However, the sensitive nature of the data and operations handled by web applications makes them susceptible to various security threats. Security is paramount in web development for the following reasons:

- **Data Protection:** Web applications often deal with sensitive user data, including personal information, financial details, and confidential documents. Ensuring data protection is vital to maintain user trust.
- **Business Continuity:** Security breaches can lead to service disruptions, financial losses, and reputational damage. Protecting against attacks helps maintain business continuity.
- **Legal and Regulatory Compliance:** Many industries have regulations (e.g., GDPR, HIPAA) that mandate the protection of user data. Non-compliance can result in significant legal consequences.
- **User Trust:** Users expect their information to be secure when interacting with web applications. A breach of trust can lead to user attrition and damage to brand reputation.

Common Security Threats

a. SQL Injection (SQLi)

Attackers inject malicious SQL queries into input fields to manipulate or retrieve unauthorized data from the database. Prevention: Use parameterized queries and prepared statements to sanitize user inputs.

b. Cross-Site Scripting (XSS)

Attackers inject malicious scripts into a web application, which are then executed by

unsuspecting users' browsers. Prevention: Sanitize and validate user inputs, and use output encoding to prevent script execution.

c. Cross-Site Request Forgery (CSRF)

Attackers trick users into performing actions without their knowledge or consent by exploiting their authenticated sessions. Prevention: Use tokens to validate and authenticate requests to ensure they originate from the legitimate user.

d. Brute Force Attacks

Attackers attempt to gain unauthorized access to accounts by trying various username and password combinations. Prevention: Implement account lockouts, CAPTCHA, and strong password policies.

e. Session Hijacking

Attackers steal users' session tokens to impersonate them and gain unauthorized access to accounts. Prevention: Use secure cookies, implement session timeouts, and enable HTTPS.

f. Security Misconfigurations

Poorly configured security settings, default credentials, and exposed sensitive information can be exploited by attackers. Prevention: Regularly review and update security configurations.

g. Sensitive Data Exposure

Inadequate protection of sensitive data such as passwords, credit card details, and personal information. Prevention: Encrypt sensitive data, follow industry standards for data protection.

h. Unvalidated Redirects and Forwards

Attackers manipulate URLs to redirect users to malicious websites or pages.
Prevention: Validate and sanitize input parameters used in URL redirection.

Understanding these common security threats is essential for web developers to build robust and secure applications. By implementing best practices and security measures, developers can mitigate these threats and provide a safer online experience for users.

2. Authentication and Authorization

Authentication and authorization are two fundamental concepts in web application security that work together to control access to resources and ensure the integrity of user interactions.

Authentication

Authentication is the process of verifying the identity of a user, ensuring that they are who they claim to be. It ensures that only authorized users are allowed to access certain parts of an application or perform specific actions. Various authentication methods can be used to verify user identity, including passwords, biometrics, tokens, and more.

1. Basic Authentication:

Users provide their credentials (username and password) with each request. Credentials are Base64-encoded and sent in the request headers. Considered less secure for sensitive applications as credentials are easily visible.

2. Form-Based Authentication:

Users provide credentials through an HTML form. The server validates credentials

and issues a session cookie upon successful authentication. Commonly used for web applications with user interfaces.

3. Token-Based Authentication:

Users receive a token upon successful authentication, which is included in subsequent requests. Stateless and suitable for distributed architectures. Commonly used with RESTful APIs and single-page applications (SPAs).

Authorization

Authorization determines what actions an authenticated user is allowed to perform within the application. It defines the user's level of access and controls their ability to interact with different parts of the application. Role-based and permission-based authorization are common methods to manage access control.

1. Role-Based Authorization:

Users are assigned roles (e.g., admin, user, moderator). Different roles have different access levels to various parts of the application. Role information is usually stored in the user's profile or database record.

2. Permission-Based Authorization:

Users are assigned specific permissions or privileges. Fine-grained control over user access to specific actions or resources. Permissions can be dynamic and assigned based on user attributes or other factors.

3. Configuring Authorization and Resource Servers

In this section, you will learn the process of setting up an Authorization Server using

Spring Boot and configuring a Resource Server to protect REST APIs using OAuth2. We will implement token-based authentication and authorization.

Step 1: Set Up an Authorization Server

- Create a Spring Boot application or use an existing one.
- Add the necessary dependencies, including "Spring Security OAuth2."

```
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-security</artifactId>
</dependency>
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-oauth2-authorization-server</artifactId>
</dependency>
```

- Configure the Authorization Server by extending **AuthorizationServerConfigurerAdapter**:

```
@Configuration
public class AuthServerConfig extends AuthorizationServerConfigurerAdapter
{
    @Override
    public void configure(ClientDetailsServiceConfigurer clients) throws
    Exception {
        clients.inMemory()
```



```

        .withClient("client")

        .secret("{noop}secret") // Use proper password encoding in
production

        .scopes("read")

        .authorizedGrantTypes("authorization_code");
    }
}

```

Step 2: Configure a Resource Server

- Create a new configuration class to configure the Resource Server:

```

@Configuration
@EnableResourceServer

public class ResourceServerConfig extends ResourceServerConfigurerAdapter {

    @Override
    public void configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests()
            .antMatchers("/api/**").authenticated();
    }
}

```

Step 3: Implement Token-Based Authentication and Authorization

- Create REST APIs in your application (e.g., /api/resource) that you want to secure.

- Use `@PreAuthorize` or `@Secured` annotations to specify access control rules for methods:

```
@RestController
@RequestMapping("/api")
public class MyController {

    @GetMapping("/resource")
    @PreAuthorize("hasAuthority('read')")
    public ResponseEntity<String> getResource() {
        return ResponseEntity.ok("Secured resource");
    }
}
```

Step 4: Test the Configuration

- Start your Spring Boot application.
- Obtain an OAuth2 access token (e.g., using the authorization code flow).
- Use the access token to call the protected API endpoint (`/api/resource`).

By following these steps, you'll configure an Authorization Server, set up a Resource Server to protect your REST APIs using OAuth2, and implement token-based authentication and authorization. Customize the configuration and access control rules to match your application's requirements.

4. JWT Tokens in Spring Boot Security

JSON Web Tokens (JWT) are a popular way to securely transmit information between parties in a compact and self-contained format. JWT tokens are often used for authentication and authorization in web applications, APIs, and microservices. In the context of Spring Boot Security, JWT tokens can be used to implement token-based authentication and authorization.

What is a JWT?

A JWT is composed of three parts:

1. Header: Contains information about the algorithm used to sign the token and the type of token it is.
2. Payload: Contains claims, which are statements about an entity (typically the user) and additional metadata.
3. Signature: Used to verify that the sender of the JWT is who it says it is and to ensure that the message wasn't changed along the way.

JWTs can be digitally signed to ensure the integrity of the information they contain. This means that even though anyone can decode the contents of a JWT, they cannot modify it without invalidating the signature.

Using JWT Tokens in Spring Boot Security

Spring Boot Security supports using JWT tokens for authentication and authorization. Here's an overview of how to use JWT tokens in a Spring Boot application:

1. Add Dependencies:

- a. Include the necessary dependencies in your project, such as "spring-boot-starter-security" and "jjwt."

2. Configure Security:

- a. Create a configuration class that extends `WebSecurityConfigurerAdapter`.
- b. Configure security settings, including allowing certain public paths and requiring authentication for others.

3. Token Generation:

- a. After successful authentication, generate a JWT token using a library like `JWT`.
- b. Include user claims (e.g., username, roles) in the payload.

4. Token Verification:

- a. Create a JWT filter to intercept incoming requests and validate the JWT token in the Authorization header.
- b. Ensure that the token's signature is valid and that the token hasn't expired.

5. Authorization:

- a. Use roles or custom claims in the JWT payload to perform authorization checks.
- b. Apply authorization rules using annotations like `@PreAuthorize`.